Software Testing Fundamentals

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

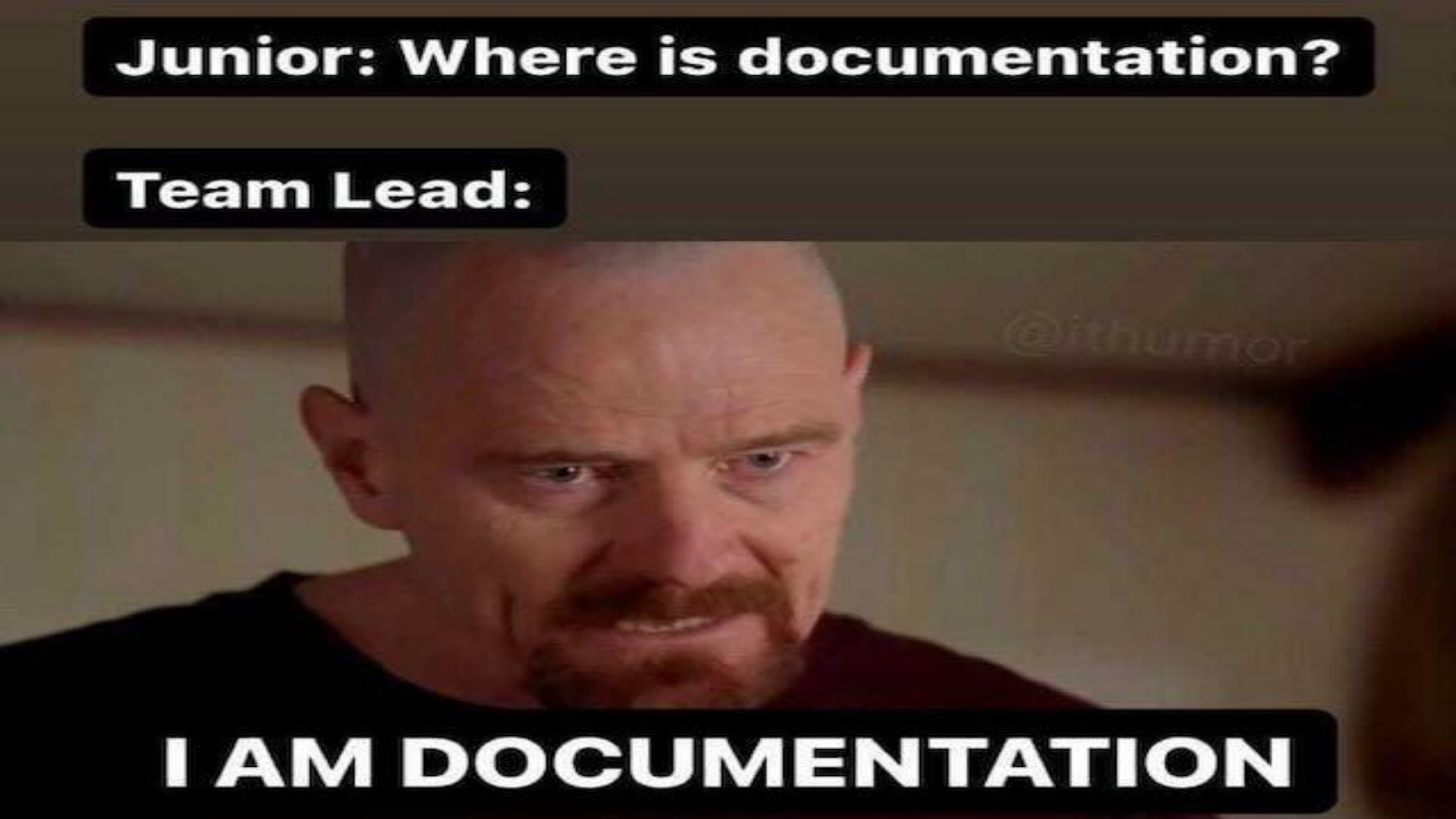
NORTHEASTERN.

Learning Objectives

- Define the purpose and value of software testing in cloud-native development
- Distinguish between different levels of testing (unit, integration, end-to-end)
- Understand the testing pyramid and its implications for test strategy
- Apply testing principles regardless of specific tools or languages

I DON'T ALWAYS TEST MY CODE
BUT WHEN I DO I DO IT IN PRODUCTION
memegenerator.net

When the bug can be called a feature


Junior: Where is documentation?
Team Lead:
@ithumor
I AM DOCUMENTATION

The entire code after I change a single letter in a comment

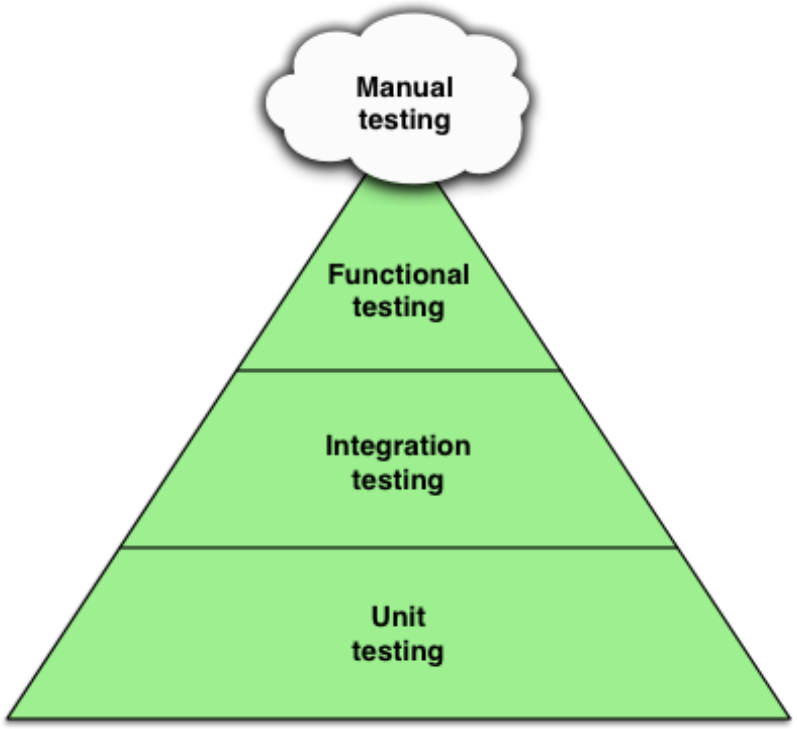
Why Test Software?

- Catch defects before they reach production
- Provide confidence during refactoring and feature development
- Serve as executable documentation of expected behavior
- Enable continuous integration and continuous deployment
- Reduce long-term maintenance costs

The Testing Pyramid

- **Base (widest):** Unit Tests — many, fast, isolated
- **Middle:** Integration Tests — moderate number, test component interactions
- **Top (narrowest):** End-to-End Tests — few, slow, test complete workflows

The pyramid optimizes for fast feedback. In CI/CD pipelines, you want tests that fail fast so developers get immediate feedback.



Unit Tests

- Tests that verify a single unit of code (function, method, class) in isolation
- Characteristics:
  - Execute in milliseconds
  - No external dependencies (database, network, file system)
  - Use test doubles (mocks, stubs, fakes) to isolate the unit under test
  - Should be deterministic—same input always produces same output

Integration Tests

- Tests that verify interactions between multiple components or systems
- What they test:
  - Database queries and transactions
  - API calls between services
  - Message queue producers and consumers
  - File system operations
  - External service integrations

Unit test vs. Integration test

End-to-End (E2E) Tests

- Tests that verify complete user workflows through the entire system
- Characteristics:
  - Simulate real user behavior
  - Test the full stack: UI → API → Database
  - Slowest to execute
  - Most prone to flakiness (non-deterministic failures)
  - Highest confidence when passing

Additional Testing Types

- **Smoke Tests:** Quick sanity checks that critical functionality works after deployment
- **Regression Tests:** Verify that new changes haven't broken existing functionality
- **Performance Tests:** Measure response times, throughput, and resource usage under load
- **Security Tests:** Identify vulnerabilities (injection, authentication flaws, etc.)
- **Contract Tests:** Verify API agreements between services

Testing in CI/CD Pipelines

Typical Pipeline Stages:

- Code pushed to repository
- Unit tests run (fail fast, < 1 minute)
- Build/compile the application
- Integration tests run (minutes)
- Deploy to staging environment
- E2E tests run against staging
- Deploy to production (if all tests pass)

Best Practices

Key principles for effective testing:

- Write tests at the appropriate level (don't over-rely on E2E)
- Keep tests fast to maintain developer productivity
- Make tests deterministic—flaky tests erode trust
- Test behavior, not implementation details
- Treat test code with the same care as production code
- Use descriptive test names that explain the expected behavior

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS